Week 4 Pre-Class Study Guide: Model Selection & Avoiding Pitfalls

Time Required: 30 minutes **Complete By:** Before Week 4 live session

---

Learning Objectives

By completing this pre-class work, you will:

- ☐ Understand why single train/test splits can be misleading
- ☐ Grasp the intuition behind cross-validation
- ☐ Recognize what regularization does (L1/L2 penalties)
- ☐ Identify what data leakage is and why it matters
- ☐ Be familiar with sklearn Pipeline concept
- ☐ Come to class ready to build production-quality ML workflows

---

Required Videos (18 minutes total)

Video 1: StatQuest - Cross-Validation (6 minutes)

Link: https://www.youtube.com/watch?v=fSytzGwwBVw

What to watch for:

- 0:00-2:00: Why we can't trust a single train/test split
- 2:00-4:00: How k-fold cross-validation works (split data k times)
- 4:00-6:00: Why CV gives more reliable performance estimates

Key Questions to Answer While Watching:

1. What problem does cross-validation solve?
2. In 5-fold CV, how many times do we train the model?
3. Why report mean ± standard deviation from CV?

Your Notes:

```
[Space for your notes]
```

Video 2: StatQuest - Ridge Regression (L2 Regularization) (12 minutes)

Link: https://www.youtube.com/watch?v=Q81RR3yKn30

What to watch for:

- 0:00-4:00: What regularization does (penalizes large coefficients)
- 4:00-8:00: How Ridge (L2) regression works
- 8:00-12:00: The lambda/alpha parameter (how much to penalize)

Key Questions to Answer While Watching:

1. What does regularization prevent?
2. How does Ridge regression differ from regular linear regression?
3. What happens when alpha is very large vs very small?

Your Notes:

`[Space for your notes]`

Required Reading (10 minutes)

sklearn Cross-validation Guide

Link: https://scikit-learn.org/stable/modules/cross_validation.html

Read these sections:

1. "Computing cross-validated metrics" (3.1)
2. "Cross-validation iterators" (3.1.1) - focus on KFold and StratifiedKFold
3. "The cross_validate function" (3.1.2.1)

What to focus on:

- The `cross_val_score()` function syntax
- What `cv=5` means (5-fold cross-validation)
- When to use StratifiedKFold (imbalanced classes)

Key concept: Cross-validation gives you a MORE RELIABLE estimate of model performance than a single train/test split.

Your Notes:

`[Space for your notes]`

Pre-Class Self-Check Quiz

Test your understanding before class. **Don't worry if you can't answer everything perfectly** - we'll cover it all in detail during the live session!

Question 1

Why is a single train/test split unreliable?

- ☐ a) It takes too long to compute
- ☐ b) Different random splits can give different performance estimates
- ☐ c) It doesn't work with decision trees
- ☐ d) sklearn doesn't support it

Click to reveal answer

Answer: b) A single train/test split is like judging a restaurant based on one meal - you might get lucky/unlucky! Different splits can give accuracy scores that vary by 3-5 percentage points just due to which samples happened to land in train vs test. Cross-validation solves this by averaging performance across multiple splits.

Question 2

In 5-fold cross-validation, how many times do we train the model?

- ☐ a) 1 time
- ☐ b) 2 times
- ☐ c) 5 times
- ☐ d) 10 times

Click to reveal answer

Answer: c) 5 times - once for each fold. The data is split into 5 parts. Each time, we use 4 parts for training and 1 part for testing. We rotate which part is the test set, training a fresh model each time. Then we average the 5 test scores.

Question 3

What does regularization do?

- ☐ a) Makes models train faster
- ☐ b) Penalizes model complexity to prevent overfitting
- ☐ c) Adds more features to the dataset
- ☐ d) Removes outliers from data

Click to reveal answer

Answer: b) Regularization adds a penalty term that discourages large model coefficients. This prevents the model from becoming too complex and overfitting to training data noise. Ridge (L2) uses squared penalty, Lasso (L1) uses absolute value penalty.

Question 4

What is a "hyperparameter"?

- ☐ a) A parameter learned from training data (like coefficients)
- ☐ b) A parameter you set BEFORE training (like max_depth or alpha)
- ☐ c) The target variable in supervised learning
- ☐ d) A very important parameter

Click to reveal answer

Answer: b) Hyperparameters are settings you choose BEFORE training begins (max_depth=5, alpha=0.1, n_estimators=100). They control the learning process but aren't learned from data. Parameters (coefficients, weights) are learned DURING training. Week 4 focuses on tuning hyperparameters systematically.

Question 5

What is data leakage?

- ☐ a) When training data accidentally includes information from the test set
- ☐ b) When models take too long to train
- ☐ c) When datasets are too small
- ☐ d) When features have missing values

Click to reveal answer

Answer: a) Data leakage occurs when information from outside the training set (like test set statistics) influences model training. Common example: scaling data before splitting - the scaler "sees" test data statistics, causing overly optimistic performance estimates. sklearn Pipeline prevents this by fitting preprocessing ONLY on training data.

Score Your Pre-Class Quiz

How many did you get right?

- **5/5:** Excellent! You're well-prepared for Week 4.
- **3-4/5:** Good foundation. Pay extra attention to concepts you missed during class.
- **1-2/5:** That's okay! The videos/reading are dense. The live session will clarify everything.
- **0/5:** No problem! Come to class ready to learn. The instructor will explain from scratch.

What to Expect in the Live Session

Segment 1 (0:00-0:15): Week 3 Recap

- Quick review of tree algorithms
- Questions from algorithm comparison homework

Segment 2 (0:15-0:45): Cross-Validation Deep Dive

- Dating analogy: "Would you marry someone after one date?"
- Live demo showing single split variance
- k-fold CV mechanics and code
- StratifiedKFold for imbalanced data

Segment 3 (0:45-1:15): Hyperparameter Tuning

- Parameters vs hyperparameters distinction
- Manual tuning problem
- GridSearchCV solution
- Interpreting `best_params_` and `cv_results_`

Segment 4 (1:15-1:30): BREAK

Segment 5 (1:30-1:55): Regularization

- L1 (Lasso) vs L2 (Ridge) comparison
- Elastic Net (combines both)
- When to use each type

Segment 6 (1:55-2:20): Data Leakage

- What is leakage and why it's dangerous
- Common leakage scenarios
- sklearn Pipeline as solution

Segment 7 (2:20-2:40): Live Coding - Complete Pipeline

- **You'll code along!** Adult Income dataset (continued from Week 3)
- Build ColumnTransformer (separate numeric/categorical preprocessing)
- Create Pipeline (preprocessing + model)
- Add GridSearchCV on top
- Save/load trained pipeline

Segment 8 (2:40-2:55): Pair Programming - "Data Leakage Detective"

- Your turn to practice with a partner
- Find and fix intentional data leakage bugs
- Refactor code to use Pipeline

Segment 9 (2:55-3:00): Wrap-up

- Production ML checklist
- Preview of Week 4 Session 2 (model interpretation)

What to Bring to Class

Required: - [ ] Laptop with Python and Jupyter installed (or Google Colab access) - [ ] sklearn library (should already be installed) - [ ] This study guide (your notes) - [ ] Week 3 knowledge (algorithms, metrics)

Optional but helpful: - [ ] Notebook/paper for additional notes - [ ] Second screen (if virtual) to have docs open alongside code

---

Tips for Success

Before the Live Session

1. Watch StatQuest videos in order (Cross-Validation, then Ridge)
2. Read sklearn cross-validation guide (focus on `cross_val_score`)
3. Complete the self-check quiz
4. Review Week 3 algorithms (you'll apply CV to them)
5. Test your Python/Jupyter environment

During the Live Session

1. Pay close attention to the "dating analogy" - it's the foundation for CV intuition
2. Ask questions during the Pipeline demo (it's a new pattern!)
3. Take notes on the four pillars: CV, GridSearch, Regularization, Pipeline
4. Code along during Segment 7 - don't just watch!
5. Engage actively during pair programming (finding bugs helps you learn)

After the Live Session

1. Complete the post-class homework (build full pipeline from scratch)
2. Review the Pipeline pattern - you'll use it in all future projects
3. Compare your solution to the provided solution notebook

Additional Resources (Optional)

If you want to dive deeper BEFORE class (completely optional):

Interactive Tutorials:

- sklearn GridSearchCV: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.htm
- sklearn Pipeline: https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.Pipeline.html
- sklearn ColumnTransformer: https://scikit-learn.org/stable/modules/generated/sklearn.compose.ColumnTransformer.

Visual Explanations:

- Cross-Validation Illustrated: https://scikit-learn.org/stable/auto_examples/model_selection/plot_cv_indices.html

- Regularization Path: https://scikit-learn.org/stable/auto_examples/linear_model/plot_ridge_path.html

Reading:

- Google ML Crash Course - Validation Set: https://developers.google.com/machine-learning/crash-course/validation/another-partition

---

Week 4 Key Terminology Preview

Get familiar with these terms - you'll use them all session!

Cross-Validation:

- **k-fold CV:** Split data into k parts, train k times using different test fold each time
- **Stratified CV:** Maintains class proportions in each fold (important for imbalanced data)
- **cross_val_score:** sklearn function that performs CV and returns scores
- **Mean ± Std Dev:** How to report CV results (e.g., "Accuracy: 0.847 ± 0.012")

Hyperparameter Tuning:

- **Parameter:** Learned from data (coefficients, weights)
- **Hyperparameter:** Set before training (max_depth, alpha, learning_rate)
- **Grid Search:** Exhaustively try all hyperparameter combinations
- **Random Search:** Randomly sample hyperparameter combinations
- **GridSearchCV:** sklearn tool that combines grid search with CV
- **best_params_:** The hyperparameter combination that gave best CV score

Regularization:

- **L1 (Lasso):** Absolute value penalty, can zero out coefficients (feature selection)
- **L2 (Ridge):** Squared penalty, shrinks coefficients toward zero
- **Elastic Net:** Combines L1 + L2 penalties
- **alpha:** Regularization strength (higher = more penalty)

Data Leakage & Pipeline:

- **Data Leakage:** Test set information leaking into training process
- **Pipeline:** sklearn tool that chains preprocessing + model
- **ColumnTransformer:** Applies different preprocessing to different columns
- **fit_transform:** Fit on training data AND transform it
- **transform:** Transform using already-fitted parameters (for test data)

Pre-Class Completion Checklist

Before the live session, make sure you:

- ☐ Watched StatQuest Cross-Validation (6 min)
- ☐ Watched StatQuest Ridge Regression (12 min)
- ☐ Read sklearn cross-validation guide (10 min)
- ☐ Completed the 5-question self-check quiz
- ☐ Reviewed Week 3 algorithms (Decision Tree, Random Forest, XGBoost)
- ☐ Tested your Python/Jupyter environment
- ☐ Reviewed learning objectives

Estimated total time: 30 minutes

Questions Before Class?

If anything is unclear, don't stress! The live session will bring it all together.

Week 4 Big Picture

What makes Week 4 different from Weeks 1-3:

- **Week 1:** Python basics, pandas, data manipulation
- **Week 2:** Data preprocessing, visualization, EDA
- **Week 3:** THREE algorithms, systematic comparison
- **Week 4:** PRODUCTION METHODOLOGY - how to build trustworthy models

Why this matters:

Week 4 is the course midpoint - you're transitioning from "learning algorithms" to "building production systems." After Week 4, you'll know:

1. How to get reliable performance estimates (CV)
2. How to tune hyperparameters systematically (GridSearchCV)
3. How to prevent overfitting (regularization)
4. How to prevent data leakage (Pipeline)

This is how professional data scientists work!

See you in class!

Last Updated: 2026-01-30